

```
/* Dirichlet-Multinomial Hierarchical Model in Stan */
```

```
data {  
  // Training data  
  int<lower=1> N; // Categories (4)  
  int<lower=1> M; // Feature dimensions (4)  
  int<lower=1> F; // Number of possible values (10)  
  // Train data  
  array[N, M, F] int<lower=0> y;  
  // Test data for the generalization task  
  array[M] int<lower=1, upper=F> x_dax;  
  array[3, M] int<lower=1, upper=F> x_choices;  
}  
  
parameters {  
  array[M] real<lower=0> alpha;  
  array[M] simplex[F] beta;  
}  
  
model {  
  alpha ~ exponential(1.0);  
  for (m in 1:M) {  
    beta[m] ~ dirichlet(rep_vector(1.0, F));  
    for (j in 1:N) {  
      // marginalized likelihood (integrate out theta)  
      y[j, m, ] ~ dirichlet_multinomial(alpha[m] * beta[m]);  
    }  
  }  
}  
  
generated quantities {  
  // Reconstruct category-specific distributions  
  array[N, M] simplex[F] theta;  
  for (m in 1:M) {  
    for (j in 1:N) {  
      theta[j, m] = dirichlet_rng(alpha[m] * beta[m] + to_vector(y[j, m, ]));  
    }  
  }  
  // Compute noun generalization choice probabilities  
  vector[3] log_prob_choice;  
  vector[3] choice_prob;  
  for (k in 1:3) {  
    log_prob_choice[k] = 0;  
    for (m in 1:M) {  
      int f_choice = x_choices[k, m];  
      real prior_count = alpha[m] * beta[m, f_choice];  
      real observed_count = (f_choice == x_dax[m]) ? 1.0 : 0.0;  
      real p = (observed_count + prior_count) / (1.0 + alpha[m]);  
      log_prob_choice[k] += log(p);  
    }  
  }  
  // Softmax normalization  
  choice_prob = softmax(log_prob_choice);  
}
```